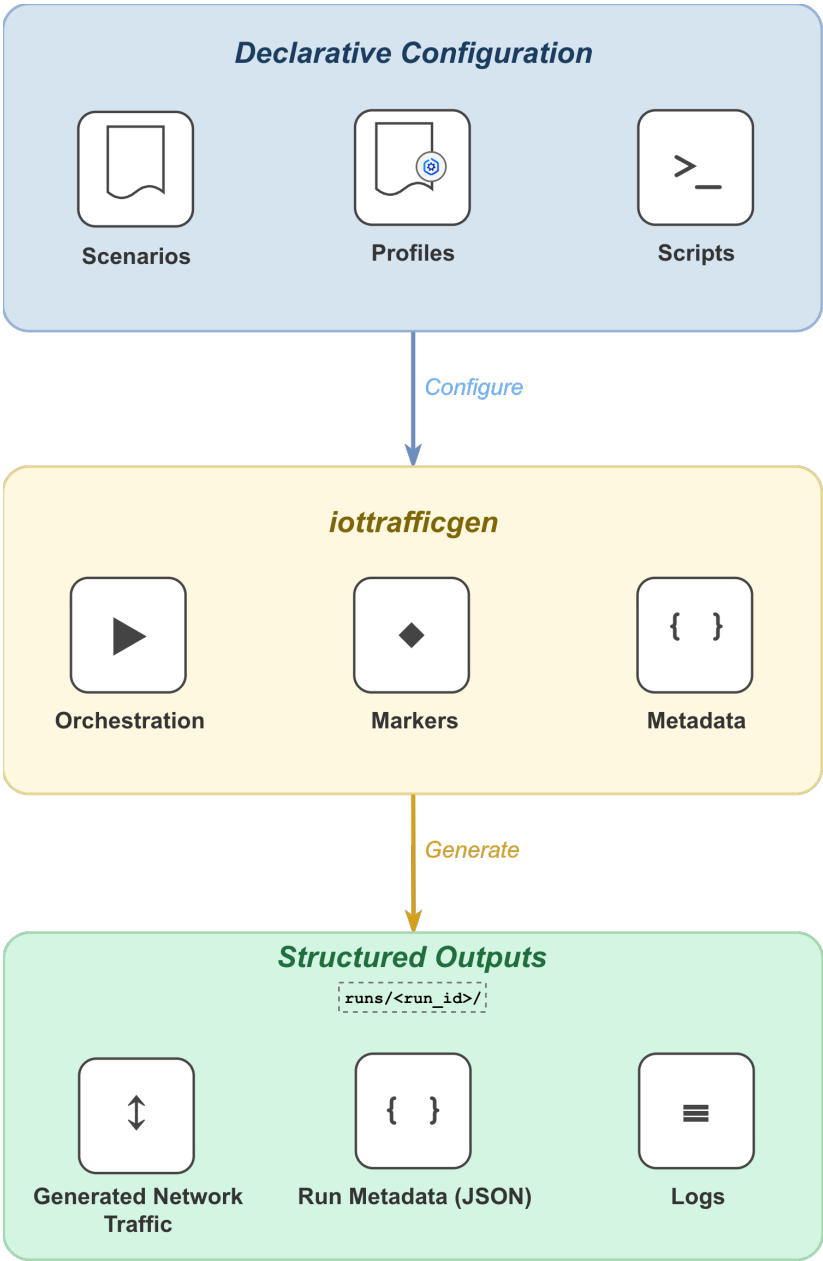


Graphical Abstract

IoT-Trafficgen: A Reproducible Network Traffic Generation Framework for IoT and Cybersecurity Research

Branly Martínez^{id}, Carlos Cambra^{id}, Álvaro Herrero^{id}, Jaime Rincón^{id}, Daniel Urda^{id}



Highlights

IoT-Trafficgen: A Reproducible Network Traffic Generation Framework for IoT and Cybersecurity Research

Branly Martínez^{}, Carlos Cambra^{}, Álvaro Herrero^{}, Jaime Rincón^{}, Daniel Urda^{}

- Open-source Python framework for reproducible IoT network traffic generation.
- Declarative YAML scenarios and reusable tool profiles separating configuration from execution.
- Automated UDP ground-truth markers for temporal alignment with packet captures.
- Structured JSON metadata per run enabling traceability and auditability.
- 66 predefined scenarios spanning seven attack categories and benign IoT traffic.

IoT-Trafficgen: A Reproducible Network Traffic Generation Framework for IoT and Cybersecurity Research

Branly Martínez^{1,*}, Carlos Cambra¹, Álvaro Herrero¹, Jaime Rincón¹,
Daniel Urda¹

^aGrupo de Inteligencia Computacional Aplicada (GICAP), Departamento de Digitalización, Escuela Politécnica Superior, Universidad de Burgos, Av. Cantabria s/n, Burgos, 09006, Spain

Abstract

Experimental cybersecurity research often depends on ad hoc traffic generation scripts and manual workflows with limited metadata, restricting reproducibility and traceability. `iottrafficgen` is an open-source Python framework that standardizes network traffic generation for IoT cybersecurity experimentation in controlled laboratory environments. The framework adopts a declarative architecture based on YAML-defined scenarios and reusable tool profiles, separating experimental definition from tool-specific configuration and execution logic. The current release provides 66 predefined scenarios and 64 tool profiles covering seven attack categories—network reconnaissance, brute force, denial of service (SYN/ICMP/MQTT flooding), SQL injection, ARP spoofing, MQTT injection, and DNS beaconing—alongside benign IoT traffic baselines. Each run produces structured JSON metadata and timestamped logs documenting resolved parameters, timing information, environment variables, and return codes. A synchronized UDP-based ground-truth marker mechanism emits boundary signals to enable precise temporal alignment with externally captured packet traces. By formalizing traffic generation workflows into version-controlled and machine-readable artifacts, `iottrafficgen` provides a reproducible and extensible software infrastructure for systematic experimentation and dataset creation in IoT cybersecurity research.

Keywords: IoT cybersecurity, Network traffic generation, Reproducible

*Corresponding author

Email addresses: bamgonzalez@ubu.es (Branly Martínez), ccbaseca@ubu.es (Carlos Cambra), ahcosio@ubu.es (Álvaro Herrero), jarincon@ubu.es (Jaime Rincón), durda@ubu.es (Daniel Urda)

experimentation, Ground-truth labelling, Declarative configuration

Metadata

Nr.	Code metadata description	Metadata
C1	Current code version	v0.1.0
C2	Permanent link to code/repository used for this code version	https://github.com/gicap-ubu/iot-trafficgen
C3	Permanent link to Reproducible Capsule	https://doi.org/10.24433/C0.1995900.v1
C4	Legal Code License	MIT License
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python 3, Bash, YAML
C7	Compilation requirements, operating environments & dependencies	Python (≥ 3.10); tested on Raspberry Pi OS and macOS. Dependencies: PyYAML (≥ 6.0), Click (≥ 8.1), Colorama ($\geq 0.4.6$), Paho-MQTT ($\geq 1.6.0$), dnspython ($\geq 2.0.0$), requests ($\geq 2.28.0$), mysql-connector-python ($\geq 8.0.0$).
C8	If available Link to developer documentation/manual	github.com/gicap-ubu/iot-trafficgen (README)
C9	Support email for questions	bamgonzalez@ubu.es

Table 1: Code metadata

1. Motivation and significance

Experimental IoT cybersecurity research frequently requires generating *labelled* network traffic under controlled conditions to evaluate defensive methods and to build reusable datasets. However, a persistent practical barrier is that traffic generation workflows are often implemented as ad hoc scripts with manual parameter tuning and informal record-keeping, which makes experiments difficult to replicate and audit across laboratories. This limitation aligns with broader reproducibility concerns reported for cybersecurity AI research and experimental networking studies [1, 2].

Public datasets such as Bot-IoT, TON_IoT, CICIoT2023, and IoT-23 have advanced benchmarking by providing labelled IoT traffic captures [3, 4, 5, 6]. Nevertheless, recent analyses highlight that dataset evolution in IoT

security remains affected by heterogeneity, protocol diversity, and lab-specific generation practices, which can limit representativeness and comparability [7]. Additionally, methodological guidance on machine learning for security emphasizes that experimental validity and documentation are critical to avoid misleading conclusions [8].

From a traffic generation standpoint, existing approaches span from replay and packet-level tools to model-driven generators [9]. In the IoT domain, IoT-Flock provides an open-source framework for IoT traffic generation [10], and Gotham offers a reproducible IoT testbed focused on security experiments and dataset generation [11]. While these efforts enable traffic production, they do not uniformly expose a lightweight orchestration model that (i) separates experiment definition from tool parameterization, (ii) records *per-execution* metadata in a structured format, and (iii) optionally emits time-aligned ground-truth signals to correlate tool execution with concurrent packet captures.

`iottrafficgen` was developed to address these limitations by providing a scenario-driven orchestration layer that wraps existing backend scripts and established security tools without requiring a specific testbed architecture. Experiments are defined declaratively through YAML scenarios and reusable tool profiles, enabling systematic parameter variation without editing scripts. Each execution automatically produces structured JSON metadata documenting resolved parameters, timing, execution outcomes, and artifact paths, together with timestamped logs. In addition, synchronized UDP ground-truth markers can be emitted during runs to enable precise temporal correlation with packet captures. By standardizing experiment definition and execution recording, `iottrafficgen` supports traceable, repeatable, and auditable IoT traffic generation for controlled laboratory research.

Table 2 summarizes representative frameworks/testbeds and dataset initiatives in relation to `iottrafficgen` across four capabilities relevant to reproducible traffic generation campaigns.

2. Software description

This section describes the architecture and core capabilities of `iottrafficgen`, including its configuration model, execution workflow, and output management.

2.1. Software architecture

Existing network traffic generators range from packet-level replay tools to statistical model-driven synthesizers [9]. Rather than generating traffic directly, `iottrafficgen` adopts a scenario-driven orchestration approach that

Table 2: Comparison of capabilities relevant to reproducible IoT traffic generation.

Framework / initiative	DS^a	RP^b	SM^c	GM^d
<i>Frameworks / testbeds</i>				
IoT-Flock [10]	No	No	No	No
Gotham [11]	Yes	Partial	Partial	No
iottrafficgen	Yes	Yes	Yes	Yes
<i>Dataset initiatives (generation-focused)</i>				
Bot-IoT [3]	No	No	Partial	No
TON_IoT [4]	No	No	Partial	No
CICIoT2023 [5]	No	No	Partial	No
IoT-23 [6]	No	No	Partial	No

^a Declarative scenarios. ^b Reusable profiles. ^c Structured metadata (per execution).

^d Ground-truth markers (time-aligned).

Note: For dataset initiatives, the assessment refers to whether the *generation process* provides reusable artifacts (not whether the released dataset includes labels/metadata).

wraps established security tools. The framework follows a layered architecture organized around four concerns: scenario definition, execution orchestration, metadata generation, and user interaction. Figure 1 illustrates the main components and their relationships.

In the configuration layer, experiments are specified using two complementary YAML file types. *Scenario files* declare what to execute: backend scripts, execution order, environment variables, and optional ground-truth marker settings. *Profile files* declare how each tool is configured, encapsulating tool-specific arguments (e.g., thread count, timeout, scan flags) independently of the scenario. This separation allows researchers to reuse the same profile across different scenarios or vary tool parameters without modifying the experimental definition.

The execution layer, implemented in the `core` module, resolves script and profile paths, validates environment variables for unconfigured placeholders, and sequentially executes each run entry. Backend scripts are invoked as subprocesses with a merged environment combining system variables, scenario-defined parameters, and profile-injected arguments (via `TOOL_ARGS`). The framework supports both time-bounded executions (controlled via `DURATION_SECONDS` with progress tracking) and open-ended runs that continue until the script terminates or the user interrupts. In both modes, SIGINT is propagated to the subprocess, allowing backend scripts to perform their own cleanup.

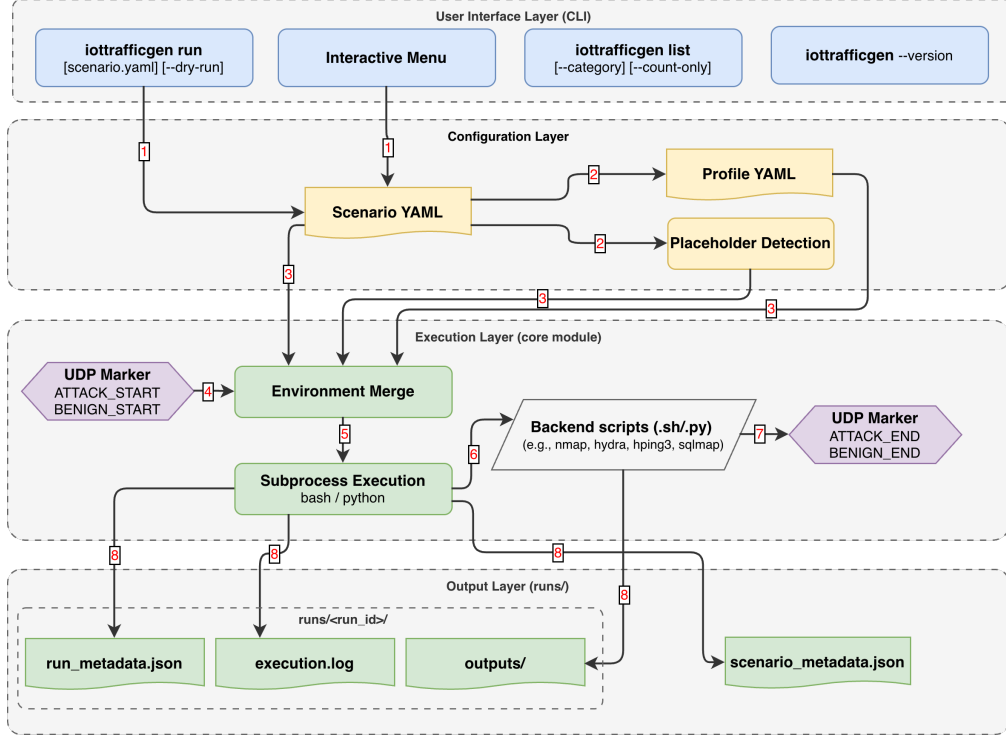


Figure 1: Software architecture of **iottrafficgen**. Numbered arrows indicate the execution flow: (1) CLI invocation, (2) configuration resolution, (3) parameter injection, (4) start marker, (5–6) script execution, (7) end marker, and (8) output generation. Dashed box groups per-run artifacts.

Adding a new traffic type requires only a backend script, a tool profile, and a scenario YAML file; the orchestration logic remains unchanged.

The metadata layer generates a structured JSON file for each run, recording a unique timestamped identifier, start and end times (UTC), execution duration, return code, resolved environment variables, and output artifact paths. For benign traffic runs, additional fields capture device count, IP address ranges, and infrastructure endpoints. A scenario-level metadata file can also aggregate results across all runs in a single execution.

The ground-truth marker subsystem emits UDP datagrams with JSON payloads at the start and end of each run. Each marker includes an event type (**ATTACK_START**, **ATTACK_END**, **BENIGN_START**, **BENIGN_END**), the run identifier, and timestamps (Unix and ISO 8601). Because the markers traverse the network as regular UDP packets, they can be captured alongside the generated traffic by any concurrent packet capture tool (e.g., **tcpdump**), enabling post-hoc temporal alignment between execution events and recorded traffic. The resulting captures can then be processed by network flow extraction

frameworks such as NFStream [12] to derive flow-level features suitable for machine learning pipelines.

2.2. Software functionalities

`iottrafficgen` exposes its functionality through a command-line interface (CLI) built with Click [13]. The CLI provides three main commands:

- `iottrafficgen run [scenario.yaml]`: Executes a scenario file. When invoked without an argument, an interactive terminal menu allows browsing categories and selecting a scenario after prompting for required parameters. The `--dry-run` flag validates the scenario without invoking backend scripts.
- `iottrafficgen list`: Displays available scenarios organized by category, with optional filtering (`--category`) or summary count (`--count-only`).
- `iottrafficgen --version`: Reports the installed framework version.

The repository ships with 66 predefined scenarios and 64 profiles organized into seven attack categories and one benign baseline category. Table 3 summarizes them.

The three benign definitions correspond to role-specific components (infrastructure check, MQTT bridge, and sensor swarm) intended to run on different laboratory nodes to form a complete background workload. Benign menu entries in interactive mode depend on the node where the CLI is invoked, since each component is bound to a specific laboratory role.

Each scenario is self-contained and references its backend script, tool profile, and all required environment variables. Placeholders (identified by the `_PLACEHOLDER` suffix) are resolved interactively before execution, displaying default values when provided in the scenario definition.

The output of each execution is organized in a timestamped directory under `runs/`:

```
runs/<run_id>/
|-- run_metadata.json
|-- execution_<timestamp>.log
\-- outputs/
```

This convention enforces per-run isolation and facilitates artifact traceability. The framework also exposes a programmatic Python API (`run_scenario`) for integration into automated pipelines without interactive input.

Table 3: Predefined scenario and baseline components included in `iottrafficgen`.

Category	Tool(s)	Scenarios	Description
Network scanning	<code>nmap</code>	30	Host discovery and port scanning
Brute force	<code>hydra</code>	6	SSH credential brute-force traffic
Denial of service	<code>hping3</code> , Python	17	SYN, ICMP, and MQTT flooding
SQL injection	<code>sqlmap</code>	6	Automated injection techniques
ARP spoofing	<code>arpspoof</code>	1	ARP cache poisoning (MITM)
MQTT injection	Python	2	False sensor data injection
DNS beaconing	Python	1	Simulated C2 via DNS queries
Benign IoT traffic	Python	3	Infrastructure check, MQTT bridge, sensor swarm
Total		66	

2.3. Defining custom scenarios

A scenario in `iottrafficgen` is a YAML document that declares an execution plan composed of one or more runs. Custom scenarios are defined without modifying the framework source code and rely on three external artifacts:

1. **Scenario YAML** (under `scenarios/`): specifies the run list and, for each run, the backend script to execute, the profile to load, and the environment variables to pass.
2. **Profile YAML** (under `profiles/`, optional): stores tool-specific arguments injected into the backend as environment variables (e.g., `TOOL_ARGS`), enabling reuse of the same tool configuration across scenarios. If no profile is referenced, the backend is executed with the environment variables defined in the scenario only.
3. **Backend script** (Bash or Python): implements the traffic workload. Scripts are parameterized exclusively via environment variables and write artifacts to `$OUT_DIR`.

Listing 1 shows a minimal scenario with a single run. The `type` field (`benign` or `attack`) is recorded in `run_metadata.json` and, when markers are enabled, determines the event labels emitted during execution. The `script` and `profile` paths are resolved relative to the scenario file. The `env` map defines the run-time parameters consumed by the backend; values suffixed with `_PLACEHOLDER` are resolved interactively by the CLI at launch time.

Listing 1: Minimal custom scenario definition with a single run.

```
scenario:
  name: "MY_CUSTOM_RUN"
  description: "Minimal example invoking a user script"

runs:
  - id: custom_01
    type: attack
    label: "CUSTOM_VARIANT_A"
    script: ../../scripts/attacks/custom/my_run.sh
    profile: ../../profiles/custom/profile_01.yaml
    env:
      TARGET_IP: "TARGET_IP_PLACEHOLDER"
      DURATION_SECONDS: "60"
      TARGET_PORT: "80"
```

Scenario files can be validated without executing any backend script using `--dry-run`, which checks YAML syntax, placeholder detection, and path resolution. A validated scenario can then be executed through the CLI or the programmatic API (`run_scenario`), producing the standard run directory, execution log, and `run_metadata.json`.

3. Illustrative Examples

This section presents two reproducible execution workflows using **iottrafficgen**: a benign infrastructure verification run and an attack scenario with ground-truth marker alignment. The section concludes with a note on programmatic invocation.

3.1. Experimental setup

The laboratory network consisted of five nodes connected to an isolated 192.168.8.0/24 subnet behind a dedicated gateway. Table 4 summarizes the equipment and their roles. The framework was installed on all Linux nodes using an editable installation, whereas external tools (hping3, nmap, hydra, and sqlmap) were deployed only on the attacker node. The traffic was passively captured on a mirrored interface.

Table 4: Laboratory equipment used in the illustrative examples.

Role	Hostname	IP Address	Platform
Attacker	RPIUBU04	192.168.8.120	Kali Linux
IoT Hub	RPIUBU02	192.168.8.121	MQTT Broker
SCADA Server	RPIUBU03	192.168.8.122	LAMP Stack
Sensor Swarm	RPIUBU05	192.168.8.123	Ubuntu
Sniffer	Desktop-PC	N/A (passive)	Windows / Wireshark

3.2. Example 1: Benign infrastructure verification

The first example runs a benign scenario used to confirm that key laboratory services required by the data-generation pipeline are available and correctly configured on the RPIUBU03 node. The scenario executes an infrastructure-check backend that reports the status of the local web stack (Apache/PHP) and the database service (MariaDB/MySQL), performs a basic database connectivity query, and validates the presence of the pre-populated **sensores** table used by the laboratory monitoring scripts.

```
$ iottrafficgen run scenarios/benign/03_infrastructure.yaml
```

In our setup, the run completed successfully in 0.22s with exit code 0. Figure 2 shows an excerpt of the console output, including per-service status messages (Apache and MariaDB running), a successful database connection, confirmation that the **sensores** table exists and contains records, and a reachable web endpoint.

```

15:13:54 [INFO] Executing script: scenarios/benign/../../../../scripts/benign/check_infrastructure.sh
15:13:54 [DEBUG] Command: bash scenarios/benign/../../../../scripts/benign/check_infrastructure.sh
[infrastructure] Checking services...
[infrastructure] Apache2: running
[infrastructure] MariaDB/MySQL: running
[infrastructure] Testing database connection...
[infrastructure] Database connection: OK
[infrastructure] Database 'iot_db': exists
[infrastructure] Table 'sensores': exists
[infrastructure] Records in 'sensores': 35
[infrastructure] Testing web endpoint...
[infrastructure] Web endpoint: OK (http://localhost/index.php?id=1&batt=50)
15:13:55 [DEBUG] Script exit code: 0
15:13:55 [INFO] Execution completed in 0.22s with code 0
15:13:55 [INFO] Sending BENIGN_END marker
→ Metadata: .....iot-trafficgen/runs/infrastructure_check_20260303_141354/run_metadat
a.json
15:13:55 [DEBUG] Saved metadata: ...../infrastructure_check_20260303_1
41354/run_metadata.json
[OK] Completed in 0.22s

```

Figure 2: Example 1 (infrastructure verification): excerpt of the console-reported results produced by the infrastructure check script, including per-service status messages and successful completion (exit code 0).

The framework also created a timestamped run directory (Figure 3) containing `run_metadata.json`, an execution log, and an `outputs/` folder. The `run_metadata.json` file records the framework version and a unique run identifier, execution type (benign), UTC timestamps, execution duration, return code, and resolved environment variables, enabling repeatable execution and auditable provenance for benign baseline runs.

```

rpiubu03@raspberrypi:~/iot-trafficgen $ tree -L 2 runs/infrastructure_check_20260303_141354/
runs/infrastructure_check_20260303_141354/
├── execution_20260303_141354.log
├── outputs
└── run_metadata.json
2 directories, 2 files

```

Figure 3: Example 1: timestamped run directory created by `iottrafficgen`, containing `run_metadata.json`, an execution log, and an `outputs/` folder.

3.3. Example 2: Attack scenario with ground-truth marker alignment

The second example demonstrates a SYN flood denial-of-service scenario executed from the attacker node (RPIUBU04) against the SCADA server (RPIUBU03, 192.168.8.122), with concurrent packet capture on the sniffer node.

Step 1: Start packet capture

On the sniffer node, a TShark capture was started on the port-mirroring interface:

```

PS> tshark -i <mirrored_if> -f "ether host <lab_nodes>"
-w SYN_FLOOD.pcapng

```

Step 2: Execute the scenario

On RPIUBU04, the framework was launched in interactive mode using `sudo iottrafficgen run`. Figure 4 shows the category selection menu. The user selected category [4] Denial of Service, then scenario [12] `syn_01`, corresponding to a high-rate SYN flood with a 120-byte payload using `hping3`.

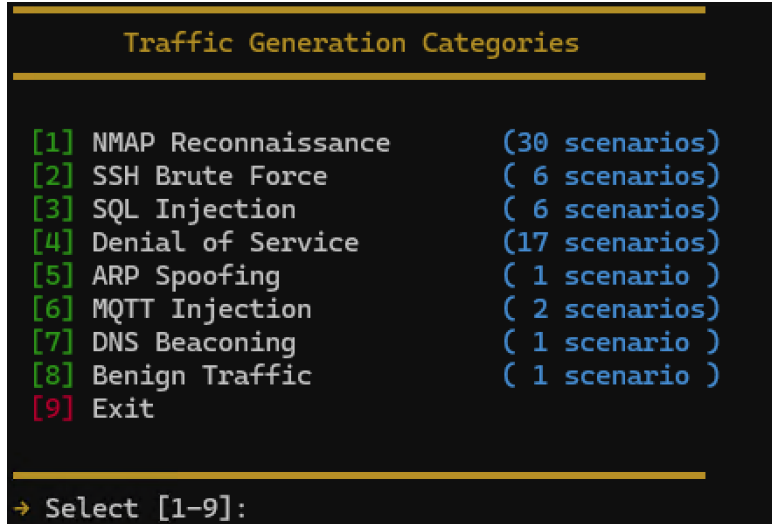


Figure 4: Interactive scenario selection menu showing the eight available traffic generation categories.

After resolving the required parameters (duration, marker host, target IP, and port), the framework emitted an `ATTACK_START` marker, executed the backend script for 60 s, and transmitted an `ATTACK_END` marker upon completion. Listing 2 shows an excerpt of the generated metadata.

Listing 2: Excerpt of `run_metadata.json` for the SYN flood scenario.

```
{
  "run_id": "syn_flood_01_20260218_143116",
  "type": "attack",
  "label": "SYN_FLOOD_HIGH_RATE_STD",
  "start_time_utc": "2026-02-18T14:31:16Z",
  "end_time_utc": "2026-02-18T14:32:16Z",
  "duration_s": 60.083,
  "returncode": 0,
  "env_effective": {
    "TARGET_IP": "192.168.8.122",
    "DURATION_SECONDS": "60",
    "TOOL_ARGS": "-d 120 -r -i u10000"
  }
}
```

}

Step 3: Ground-truth marker alignment

After stopping the capture (72471 packets), the ground-truth markers were extracted by filtering on UDP port 55556. Figure 5 shows the Wireshark display with the two marker datagrams (ATTACK_START at $t = 72.25$ s and ATTACK_END at $t = 132.33$ s).

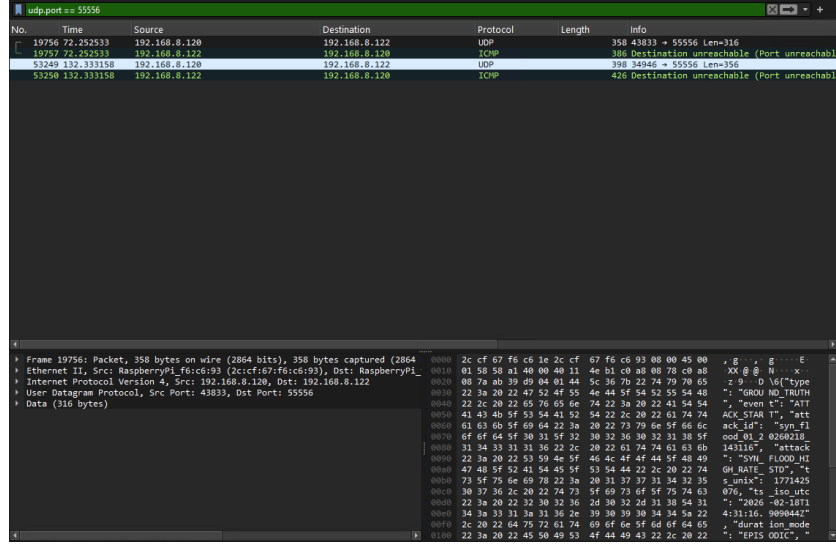


Figure 5: Wireshark capture filtered by `udp.port == 55556`, showing the ATTACK_START and ATTACK_END markers with their timestamps and payload lengths.

The time difference (60.08 s) is consistent with the configured duration and the recorded `duration_s` in the metadata. Each marker payload contains the event type, run identifier, attack label, target IP, and ISO 8601 timestamp. The `ts_iso_utc` field in the captured marker is consistent with `start_time_utc` in the run metadata (Listing 2), confirming the temporal consistency between in-network markers and execution artifacts. This alignment enables researchers to label individual packets by cross-referencing the timestamps with the marker intervals, thereby eliminating manual labeling ambiguity.

3.4. Programmatic invocation

In addition to the CLI, the framework exposes a Python API (`run_scenario`) that accepts the same parameters (scenario path, workspace, dry-run, verbosity) and produces the same artifacts. This interface enables integration into batch workflows and automated pipelines without requiring interactive input.

4. Impact

`iottrafficgen` targets a recurring bottleneck in experimental IoT security research: generating *labeled* network traffic under controlled laboratory conditions in a way that is portable, repeatable, and auditable. While public datasets (e.g., CIC-IDS, IoT-23, Edge-IIoTset [14]) enable benchmarking, researchers who need traffic tailored to custom topologies, device roles, or parameter sweeps often rely on ad-hoc scripts that provide limited standardization, weak provenance, and incomplete ground truth. The backend traffic-generation scripts consolidated by `iottrafficgen` were originally used to support the generation of the publicly available labeled IoT traffic dataset reported in [15].

The framework introduces a declarative workflow that separates (i) experimental definition (scenario YAML files), (ii) tool parameterization (profile YAML files), and (iii) execution backends (shell/Python scripts). The current release provides 66 scenario YAML files and 64 reusable tool profiles spanning reconnaissance, brute force, SQL injection, denial-of-service, man-in-the-middle, MQTT injection, DNS beaconing, and benign IoT baseline generation. This structure supports systematic parameter variation without editing scenario files or backend scripts, facilitating replication across laboratories and longitudinal comparisons across experiments.

For each run, `iottrafficgen` produces machine-readable execution records (JSON metadata and logs) that capture a unique run identifier, UTC timestamps, execution duration, resolved environment variables, return codes, and output artifact paths. This metadata layer enables independent verification of experimental conditions and improves traceability when constructing datasets from packet captures. In addition, scenarios can optionally emit lightweight UDP temporal markers to delimit benign/attack-pattern intervals, supporting consistent alignment between traffic traces and ground-truth annotations. Portability is provided at the orchestration level, assuming required external tools are available in the target environment.

By packaging these elements into a version-controlled software artifact, `iottrafficgen` provides a tool-agnostic orchestration layer that complements existing IoT laboratory infrastructures without imposing a specific topology or emulation environment. Unlike protocol-specific traffic generators or full-scale testbed platforms, the framework centers on declarative experiment definition combined with structured execution metadata and in-network ground-truth signaling.

This design enables post-hoc independent reconstruction of experimental conditions directly from the generated artifacts, reducing ambiguity in dataset construction workflows and strengthening methodological transparency in

IoT cybersecurity experimentation. The metadata and marker subsystems operate with negligible overhead relative to the underlying traffic-generation tools, preserving the integrity of the generated traffic patterns while improving auditability and reproducibility.

5. Conclusions

This paper introduced `iottrafficgen`, an open-source Python framework designed to standardize and structure IoT network traffic generation under controlled laboratory conditions. By combining declarative YAML-based scenario definitions, reusable tool profiles, automated UDP ground-truth markers, and structured per-run metadata, the framework provides a reproducible and traceable experimental infrastructure for IoT cybersecurity research.

The current release comprises 66 predefined scenarios and 64 tool profiles spanning reconnaissance, brute force, denial-of-service, injection techniques, man-in-the-middle, command-and-control simulation, and benign IoT traffic generation. The framework has been exercised and demonstrated in an isolated Raspberry Pi-based laboratory environment and consolidates backend scripts previously used in laboratory data collection campaigns supporting machine learning-based cyberattack detection studies.

Beyond traffic dataset construction workflows, `iottrafficgen` establishes a structured software foundation for controlled experimentation, enabling systematic variation of scenario parameters while preserving execution consistency and auditability. As a version-controlled and extensible research artifact, it contributes to methodological rigor and reproducibility in experimental IoT cybersecurity and AI-driven network traffic analysis.

By abstracting traffic generation into declarative configurations and structured execution artifacts, `iottrafficgen` shifts laboratory traffic campaigns from ad hoc scripting practices toward software-engineered experimental workflows. This abstraction enables reproducible traffic studies across different laboratories without imposing a fixed topology or emulation environment.

Limitations

`iottrafficgen` relies on external traffic-generation tools (e.g., `nmap`, `hping3`, `hydra`) that must be installed and configured by the user. Some traffic generation scenarios (e.g., SYN flooding, ARP spoofing) may require elevated privileges and generate traffic patterns that are intended only for isolated laboratory environments. The current version executes runs sequentially on a single host; parallel or distributed orchestration is out of scope. While the `--dry-run` mode validates configuration and parameter resolution, it cannot guarantee the runtime availability of all external dependencies. End-to-end

verification (e.g., confirming that traffic reached the intended target or was captured) depends on the experimental testbed and capture setup and is therefore outside the scope of the framework.

The source code is available under the MIT license at <https://github.com/gicap-ubu/iot-trafficgen>.

CRedit authorship contribution statement

Branly Martínez: Writing – original draft, Software, Visualization, Conceptualization, Methodology, Investigation, Validation.

Carlos Cambra: Writing – review and editing, Supervision.

Jaime Rincón: Writing – review and editing, Supervision.

Daniel Urda: Writing – review and editing, Supervision, Conceptualization, Validation.

Álvaro Herrero: Supervision, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used ChatGPT to proofread and improve the language. After using this tool/service, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the published article.

Acknowledgements

This publication is part of the AI4SECIoT project ("Artificial Intelligence for Securing IoT Devices"), funded by the National Cybersecurity Institute (INCIBE), derived from a collaboration agreement signed between the National Institute of Cybersecurity (INCIBE) and the University of Burgos. This initiative is carried out within the framework of the Recovery, Transformation and Resilience Plan funds, financed by the European Union (Next Generation), the project of the Government of Spain that outlines the roadmap for the modernization of the Spanish economy, the recovery of economic growth and job creation, for solid, inclusive and resilient economic reconstruction after the COVID19 crisis, and to respond to the challenges of the next decade.

References

- [1] R. H. Moulton, G. A. McCully, J. D. Hastings, Confronting the reproducibility crisis: A case study of challenges in cybersecurity AI (2024). [arXiv:2405.18753](#).
- [2] V. Bajpai, A. Brunstrom, A. Feldmann, W. Kellerer, A. Pras, H. Schulzrinne, G. Smaragdakis, M. Wählisch, K. Wehrle, The Dagstuhl beginners guide to reproducibility for experimental networking research, *ACM SIGCOMM Computer Communication Review* 49 (1) (2019) 24–30. [doi:10.1145/3314212.3314217](#).
- [3] N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull, Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset, *Future Generation Computer Systems* 100 (2019) 779–796. [doi:10.1016/j.future.2019.05.041](#).
- [4] N. Moustafa, A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets, *Sustainable Cities and Society* 72 (2021) 102994. [doi:10.1016/j.scs.2021.102994](#).
- [5] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zúquete, G. Body, A. Shami, A. A. Ghorbani, CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment, *Sensors* 23 (13) (2023) 5941. [doi:10.3390/s23135941](#).
- [6] S. Garcia, A. Parmisano, M. J. Erquiaga, IoT-23: A labeled dataset with malicious and benign IoT network traffic (2020). [doi:10.5281/zenodo.4743746](#).
- [7] B. Kaur, S. Dadkhah, F. Shoeleh, E. C. P. Neto, P. Xiong, S. Iqbal, P. Lamontagne, S. Ray, A. A. Ghorbani, Internet of Things (IoT) security dataset evolution: Challenges and future directions, *Internet of Things* 22 (2023) 100780. [doi:10.1016/j.iot.2023.100780](#).
- [8] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, K. Rieck, Dos and don'ts of machine learning in computer security, in: *31st USENIX Security Symposium (USENIX Security '22)*, Boston, MA, 2022, pp. 3971–3988.
- [9] O. A. Adeleke, N. Bastin, D. Gurkan, Network traffic generation: A survey and methodology, *ACM Computing Surveys* 55 (2) (2022) 1–23. [doi:10.1145/3488375](#).

- [10] S. Ghazanfar, F. Hussain, A. U. Rehman, U. U. Fayyaz, F. Shahzad, G. A. Shah, IoT-Flock: An open-source framework for IoT traffic generation, in: Proceedings of the 2020 International Conference on Emerging Trends in Smart Technologies (ICETST), 2020, pp. 1–6. doi:10.1109/ICETST49965.2020.9080732.
- [11] X. Sáez-de Cámara, J. L. Flores, C. Arellano, A. Urbietta, U. Zurutuza, Gotham testbed: A reproducible IoT testbed for security experiments and dataset generation, IEEE Transactions on Dependable and Secure Computing 20 (6) (2023) 4823–4840. doi:10.1109/TDSC.2023.3247166.
- [12] Z. Aouini, A. Pekar, NFStream: A flexible network data analysis framework, Computer Networks 204 (2022) 108719. doi:10.1016/j.comnet.2021.108719.
- [13] A. Ronacher, Click: A python package for creating command line interfaces, version 8.1 (2024).
URL <https://click.palletsprojects.com/>
- [14] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, H. Janicke, Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning, IEEE Access 10 (2022) 40281–40306. doi:10.1109/ACCESS.2022.3165809.
- [15] B. A. Martínez González, C. Cambra Baseca, D. Urda Muñoz, J. A. Rincón Arango, Á. Herrero Cosío, Labelled iot flow-based network traffic dataset for cyberattack detection, Research dataset (2026). doi:10.71486/dyxt-2r24.
URL <https://hdl.handle.net/10259/11273>